
EnzymeLM: Chemistry-Native Language Modeling for Balanced Enzymatic Reaction Prediction

Tanmay Ambadkar, Mohit Anand, Yu-Wei Su, Costas Maranas, Abhinav Verma
Pennsylvania State University
University Park, PA 16802
{tsa5252, mohit, yfs5313, cdm8, verma}@psu.edu

Abstract

Enzymatic reaction prediction is central to biocatalysis and biosynthetic pathway design, but current language models fail to predict complete enzymatic product sets. We show that open-source frontier LLMs, even with few-shot prompting, and chemistry-specific models perform poorly on ECRect forward prediction without EC-number input, with the strongest baseline reaching only 7.59% any-product and 1.90% all-products accuracy. We introduce EnzymeLM, a 1B-parameter chemistry-native language model that extends a pretrained Gemma tokenizer with SELFIES and reaction-specific tokens, preserving linguistic priors while aligning the token space with molecular structure. EnzymeLM is pretrained on molecule-level tasks from MetaNetX and fine-tuned on augmented ECRect reactions to generate structured reaction traces containing reaction-class hypotheses, transformation descriptions, product drafts, atom inventories, balance checks, self-correction steps, and final product SELFIES. EnzymeLM achieves 72.70% any-product accuracy, 34.80% all-products accuracy, 99.60% validity, and 0.7707 average Tanimoto similarity. Ablations show that structured reasoning improves complete product recovery, while chemistry-native tokenization and molecular pretraining are most important for full product-set prediction. These results suggest that balanced enzymatic reaction prediction is substantially improved by domain-aligned tokenization and training objectives, and that parameter scale alone is insufficient when models are not adapted to molecular structure and biochemical reasoning.

1 Introduction

Large language models (LLMs) have become powerful tools for scientific reasoning, with applications spanning medical question answering, protein structure prediction, autonomous chemical experimentation, and tool-augmented synthesis planning [1, 2, 3, 4, 5]. Chemistry is especially well suited to language modeling because molecules and reactions can be represented as sequences. However, sequence modeling alone is not sufficient for enzymatic chemistry: a useful model must recover not only plausible products, but complete, stoichiometrically balanced product sets.

Enzymatic reactions are central to biocatalysis, metabolic engineering, and biosynthetic pathway design. They often proceed under mild conditions, exhibit high stereo- and regioselectivity, and can reduce the number of synthetic steps compared with traditional chemical routes [6, 7]. Prior work has extended data-driven reaction prediction to biocatalysis using ECRect and Molecular Transformer-style architectures [8], while more recent resources such as EnzymeMap improve enzymatic reaction curation, atom mapping, and validation [9]. Nevertheless, existing enzymatic reaction models often rely on EC-number conditioning, focus on primary products, or do not explicitly enforce recovery of the complete product set. In this work, we study the harder setting where the model receives only reactant molecular strings at test time, without reactant names or EC numbers, and must generate the full enzymatic product set. Rather than treating enzymatic prediction as direct sequence translation,

we frame it as structured reaction understanding for balanced product generation: the model infers a reaction-class hypothesis, drafts candidate products, checks atom inventories, and then emits the final product set.

A central challenge is molecular representation. SMILES is widely used for molecular language modeling [10], but it is not unique and random perturbations often produce invalid molecules. SELFIES addresses this by providing a grammar-based molecular string representation in which every valid SELFIES string decodes to a molecule [11]. However, standard LLM tokenizers fragment SELFIES into generic subword pieces that do not correspond to atoms, bonds, branches, rings, or stereochemical units. This creates a representation mismatch: the model retains natural-language knowledge, but its molecular inputs are not aligned with chemically meaningful tokens.

We introduce EnzymeLM, a 1B-parameter chemistry-native language model for balanced enzymatic forward prediction. EnzymeLM extends a pretrained Gemma tokenizer with SELFIES and reaction-specific tokens, preserving pretrained linguistic priors while enabling molecular strings to be processed through chemistry-aligned vocabulary. The model is then trained in two stages. First, it is pretrained on one million MetaNetX molecules [12] using molecule-level tasks that teach SELFIES equivalence, name-to-structure mapping, and molecular formula prediction. Second, it is fine-tuned on augmented ECRreact reactions using structured biochemical reasoning traces, atom inventories, degraded self-correction examples, and mixed product-generation targets.

Empirically, current open-source LLMs and chemistry-specific models perform poorly on this task. Even with few-shot prompting and a strong system prompt, open-source LLMs up to 80B parameters fail to recover complete enzymatic product sets. The strongest chemistry-specific baseline in our evaluation, ReactionT5v2, reaches 7.59% any-product accuracy and 1.90% all-products accuracy. EnzymeLM achieves 72.70% any-product accuracy, 34.80% all-products accuracy, 99.60% validity, and 0.7707 macro Tanimoto similarity. These results show that current prompted general-purpose LLMs are insufficient for balanced enzymatic reaction prediction, and that domain-aligned tokenization and training objectives are critical when models are not specifically adapted to molecular structure and biochemical reasoning.

Contributions. We make three main contributions:

- **Chemistry-native tokenizer extension.** We extend a pretrained Gemma-3-1B tokenizer with SELFIES and reaction-specific tokens, allowing the model to preserve natural-language priors while processing molecular and reaction strings through chemically meaningful vocabulary.
- **Molecule-level pretraining for stoichiometric reasoning.** We construct a MetaNetX-derived pretraining corpus and train on SELFIES enumeration, name-to-SELFIES prediction, and formula prediction, giving the model practice with molecular equivalence, name-structure alignment, and atom-level composition.
- **Structured reaction-trace supervision for balanced product prediction.** We fine-tune EnzymeLM on augmented ECRreact reactions to generate reaction-class hypotheses, transformation descriptions, product drafts, atom inventories, balance checks, and final product SELFIES. The final product set is evaluated with strict canonical matching, while the intermediate trace provides a diagnostic scaffold for inspecting product-set failures.

2 Related Work

Language models for chemistry. General-purpose LLMs such as GPT-4 [1] and the Llama [13], Gemma [14], and Qwen [15] families demonstrate strong text-based reasoning, but they are not designed to represent molecular structure or reaction constraints natively. Chemistry-focused pretrained models such as ChemBERTa [16], MolFormer [17], and MolXPT [18] learn from large molecular corpora, often using SMILES strings, and are primarily used for molecular representation, property prediction, or general chemical sequence modeling. More recent instruction-tuned chemistry models, including ReactionT5v2 [19] and LlaSMol [20], target chemical tasks more directly. However, balanced enzymatic forward prediction requires more than producing a plausible molecule: the model must infer the transformation and recover the complete product set. Our experiments therefore evaluate whether these models transfer to the stricter no-EC, full-product enzymatic setting.

Molecular string representations and tokenization. SMILES [10] remains the dominant molecular string representation, but it is non-unique and fragile under perturbation. SELFIES [11] addresses this by defining a grammar-based representation in which valid strings decode to valid molecules. Prior work has explored molecular tokenization from several directions. Kalamkar et al. [21] augment a pretrained LLM vocabulary with chemically salient tokens learned by BPE over SMILES corpora, demonstrating the value of domain-aligned tokenization for molecular property prediction. APE [22] compares SMILES and SELFIES tokenization in chemical language modeling, while Smirk and Smirk-GPE [23] focus on OpenSMILES-compatible tokenization for molecular foundation models. SELFormer [24] trains a SELFIES-based language model from scratch, whereas UniMoT [25] introduces molecule tokens through vector quantization and a multimodal bridge. SmlSelf [26] uses SELFIES as a post-hoc correction mechanism to improve molecular validity. EnzymeLM differs from these approaches by extending a pretrained autoregressive LLM tokenizer with SELFIES grammar tokens, preserving natural-language priors while making molecular inputs chemically meaningful for reaction-level prediction.

Enzymatic reaction datasets and prediction tasks. ECRReact [8] and Molecular Transformer-based biocatalysed synthesis planning [27] introduced large-scale data-driven modeling for enzymatic forward and retrosynthetic prediction, using EC-number information to represent enzyme-class context. EnzymeMap [9] improves enzymatic reaction resources through curation, atom mapping, validation, and benchmarking for forward prediction, retrosynthesis, and regioselectivity; it is directly relevant to future evaluation on curated, atom-mapped enzymatic reactions. ReactZyme [28] frames enzyme-reaction prediction as a retrieval problem over enzymes and reactions, while RXNRECEr [29] uses protein language models for fine-grained enzymatic function annotation. Qian et al. [30] study enzyme-function prediction from enzymatic reactions. These works are complementary to our setting: they often use enzyme, EC, protein, or retrieval-style inputs, whereas EnzymeLM evaluates reactant-structure-only forward prediction at test time.

Retrosynthesis, pathway planning, and stoichiometric constraints. Several recent systems address adjacent enzymatic synthesis problems. Enzyformer [31] studies enzymatic retrosynthesis, while synthetic-potential-guided chemoenzymatic planning [32] and minChemBio [33] target multi-step pathway discovery. These tasks are downstream of, but distinct from, single-step forward product-set prediction. Stoichiometric consistency has also been addressed through rule- and optimization-based approaches such as novoStoic2.0 [34], which uses reaction rules and mixed-integer linear programming to construct balanced biochemical reactions. In contrast, EnzymeLM learns a single language-model pipeline that combines SELFIES-native tokenization, molecule-level pretraining, structured reaction-trace supervision, and direct product-generation objectives for balanced enzymatic forward prediction.

3 Method

EnzymeLM is a chemistry-native language model for enzymatic forward reaction prediction. The goal is to preserve the linguistic and scientific priors of a pretrained LLM while adapting its token space and training objectives to molecular structure, atom accounting, and balanced product-set generation. As shown in Figure 1, the pipeline has four stages: chemistry-native tokenizer extension, molecule-level pretraining, ECRReact reasoning and augmentation, and reaction-level supervised fine-tuning.

3.1 Chemistry-Native Tokenizer Extension

The first stage extends the tokenizer of a pretrained Gemma-3-1B model with chemistry-specific tokens. We retain the original tokenizer vocabulary and model weights so that the model preserves pretrained natural-language and scientific knowledge. We then add dedicated vocabulary entries for SELFIES symbols, allowing molecular structures to be represented through chemically meaningful units rather than generic subword fragments.

This extension addresses a representation mismatch. Enzymatic reaction prediction requires the model to operate over both natural language and molecular strings: biochemical descriptions, EC-class terminology, product names, and atom-balance explanations are written in English, while molecular structures are written in SELFIES. A SELFIES symbol such as [C], [=O], or [Branch1] corresponds to a chemically meaningful unit, but a standard language tokenizer may split it into

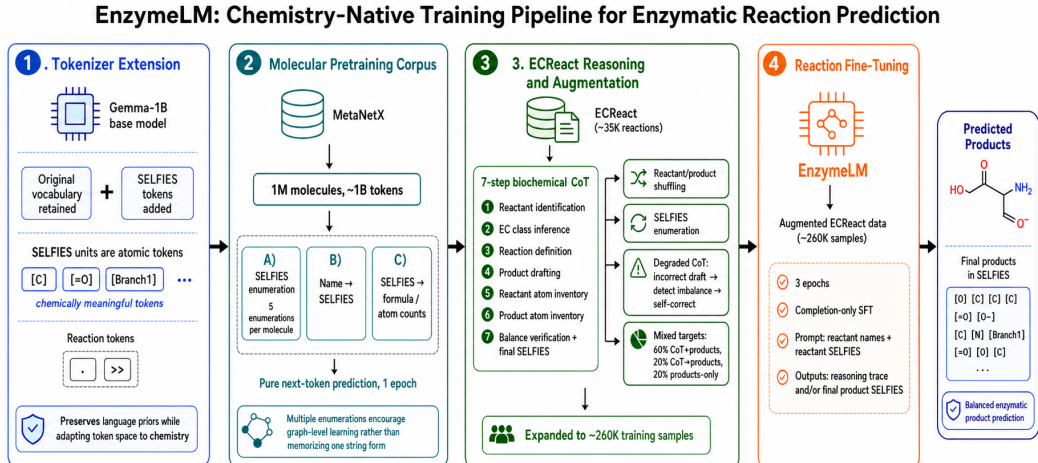


Figure 1: Overview of the EnzymeLM training pipeline for enzymatic reaction prediction. EnzymeLM extends a pretrained Gemma-3-1B tokenizer with SELFIES and reaction-specific tokens, constructs a molecule-level pretraining corpus from MetaNetX, and pretrains on chemistry-native sequence tasks using pure next-token prediction. The model is then fine-tuned on augmented ECRéact reactions using structured biochemical reasoning traces, degraded self-correction examples, reactant/product shuffling, SELFIES enumeration, and a mixed supervision curriculum. This design preserves the linguistic priors of the base model while adapting its token space and training objectives to molecular structure, stoichiometric reasoning, and balanced enzymatic product prediction.

arbitrary pieces. Tokenizer extension aligns molecular strings with the model’s token space while preserving the language priors needed for biochemical reasoning.

Let $\mathcal{V}_{\text{base}}$ denote the original Gemma tokenizer vocabulary and $\mathcal{V}_{\text{SELFIES}}$ denote the SELFIES symbols observed in the molecular pretraining corpus. We define the chemistry-extended vocabulary as

$$\mathcal{V}_{\text{chem}} = \mathcal{V}_{\text{base}} \cup \mathcal{V}_{\text{SELFIES}}.$$

For reaction fine-tuning, we further add reaction-formatting tokens such as the molecular separator $.$ and the reaction arrow $\gg$. The final reaction vocabulary is

$$\mathcal{V}_{\text{rxn}} = \mathcal{V}_{\text{chem}} \cup \mathcal{V}_{\text{reaction}}.$$

The corresponding token embeddings are added to the model embedding matrix and trained during molecular pretraining and reaction fine-tuning.

3.2 Molecule-Level Pretraining

The second stage adapts the tokenizer-extended model to molecular structure before exposing it to enzymatic reactions. We construct a molecule-level pretraining corpus from MetaNetX containing one million molecules with molecular names, SMILES strings, molecular formulae, and structural annotations. SMILES strings are parsed with RDKit and converted into SELFIES; atom counts are computed from RDKit molecular graphs; and molecules without valid structural conversion are removed.

Each molecule is represented as

$$m = (n, s, f, a),$$

where n is the molecular name, s is a SELFIES representation, f is the molecular formula, and a is the atom-count dictionary.

From this corpus, we generate three task families. The first is SELFIES-to-SELFIES enumeration, where one valid SELFIES string is mapped to another valid SELFIES string for the same molecule. This teaches that molecular identity is graph-based rather than tied to a single string form. The second is name-to-SELFIES prediction, which grounds chemical nomenclature in molecular structure. The

third is SELFIES-to-formula prediction, which gives the model practice with molecular composition and atom-level accounting.

For each molecule, the preprocessing pipeline generates up to five distinct SELFIES enumerations. These enumerations are used across the molecule-level task families so that the model is not trained to parrot a single canonical representation. The resulting pretraining corpus contains approximately one billion tokens. EnzymeLM is pretrained for one epoch using pure autoregressive next-token prediction over the serialized task sequences, without completion-only masking or task-specific loss masking.

This stage is not intended to teach enzymatic reaction prediction directly. Its purpose is to give the model a chemistry-native representation space, so that later reaction fine-tuning can build on molecular equivalence, name-structure alignment, and atom-counting ability.

3.3 ECRReact Reasoning and Reaction Augmentation

The third stage adapts EnzymeLM from molecule-level structure learning to enzymatic reaction reasoning. We use ECRReact as the reaction source. After deduplication, the dataset contains approximately 35,000 unique enzymatic reactions, split into approximately 33,000 training reactions, 1,000 validation reactions, and 1,000 test reactions. Reactant and product SMILES are converted into SELFIES, and reactions with invalid molecular conversion are removed.

A key distinction between training and evaluation is input format. During SFT data generation, prompts can be either hybrid prompts, containing compound names and reactant SELFIES, or structure-only prompts, containing only reactant SELFIES. In all reported test-time evaluation, the model receives structure-only prompts: reactant SELFIES are provided, but reactant names and EC numbers are withheld. EC annotations are used only to construct target reasoning traces during training, so the model must learn to infer EC-informed reaction structure rather than receiving the EC number as input.

We generate the SFT corpus through an automated pipeline grounded in RDKit, SELFIES, ECRReact metadata, MetaNetX compound names, and EC hierarchy information. Compound names are recovered through MetaNetX lookup where available. Atom inventories are computed from decoded molecular graphs using RDKit with explicit hydrogens, allowing mass balance to be verified from structure rather than from free-form text.

To construct EC-informed reasoning traces, reactions are mapped to Enzyme Commission classes through a cascaded lookup strategy. The mapping first attempts exact reactant-product set matches, then core-molecule matches, largest reactant-product pair matches, and single-substrate matches. When no specific EC lookup is available, a heuristic Level 1 EC class is assigned. The resulting EC information is used to construct a natural-language classification and, when available, a mechanism description from EC hierarchy metadata and class-specific templates.

Each generated trace contains the following fields:

1. **Reactant identification:** identify the reactants represented by the input structures.
2. **Reaction classification:** infer an enzymatic class.
3. **Reaction definition:** describe the expected biochemical transformation.
4. **Product drafting:** propose expected product names.
5. **Reactant atom inventory:** list atom counts for the reactant side.
6. **Product atom inventory:** list atom counts for the product side.
7. **Balance verification:** check whether the predicted product side is atom-balanced.
8. **Final prediction:** emit the final product SELFIES.

The trace exposes the model’s intermediate reaction-class, product-drafting, and atom-accounting steps before final product generation. The EC class appears only in the target trace, not in the input prompt.

We expand the ECRReact training set to approximately 260,000 samples through reaction-level augmentation. Each training reaction is expanded into ten augmented samples using randomized

SELFIES enumeration and reactant/product permutation. Prompt format is sampled with a 50/50 mixture: hybrid prompts contain compound names and reactant SELFIES, while structure-only prompts contain only reactant SELFIES. This teaches the model to connect names to structures during training while remaining robust to the purely structural inputs used in the reported evaluation.

To train self-correction, 20% of generated reasoning traces include deliberate atom-inventory perturbations. In these degraded traces, the product inventory is first corrupted, the reaction is marked as imbalanced, the atom-count delta is reported, and a correction step restores the correct inventory before the final product SELFIES:

draft products $\rightarrow$ incorrect inventory $\rightarrow$ imbalance detected $\rightarrow$ corrected inventory $\rightarrow$ final SELFIES.

This makes atom balance an operational constraint rather than decorative text. The model is trained not only to emit final products, but also to recognize and correct intermediate stoichiometric inconsistencies. The trace turns product generation into a structured prediction problem: before emitting final SELFIES, the model must generate intermediate claims about reaction class, transformation type, product identities, atom inventories, and balance status. We treat these intermediate fields as diagnostic outputs. The primary quantitative endpoint remains the final canonicalized product set.

3.4 Reaction Fine-Tuning

The final stage fine-tunes EnzymeLM on the augmented ECR react corpus for three epochs using completion-only supervised fine-tuning. Each example is represented as a prompt paired with a target response. Prompt tokens are excluded from the loss, and optimization is applied only to the generated target portion.

The SFT corpus uses a 60/20/20 mixture of target formats:

- **60% prompt-to-reasoning-and-products:** the model generates the full reasoning trace followed by final product SELFIES.
- **20% prompt-plus-reasoning-to-products:** the reasoning trace is included in the prompt, and the model generates only final product SELFIES.
- **20% prompt-to-products-only:** the model directly generates final product SELFIES.

This mixture separates reasoning as a training scaffold from product prediction as the final task. The first format teaches explicit biochemical reasoning, the second teaches the model to condition product generation on a reasoning trace, and the third prevents overdependence on verbose reasoning syntax. The final objective remains balanced product-set generation.

Training uses LoRA, bf16 mixed precision, gradient checkpointing, and distributed training. In the main SFT run, we use 8 processes with batch size 4 per process and gradient accumulation of 4. The model is initialized from the molecule-pretrained EnzymeLM checkpoint and trained with the chemistry-extended tokenizer. Additional implementation details are provided in Appendix B.

3.5 Design Rationale

The pipeline is motivated by two design choices. First, we extend a pretrained LLM instead of training a SELFIES-only model from scratch. Enzymatic reaction prediction requires the model to connect molecular strings with chemical names, EC-class terminology, biochemical transformations, and atom-balance explanations. Training from scratch on SELFIES alone would discard the linguistic and scientific priors already present in the pretrained Gemma model.

Second, we do not rely on reaction-only supervised fine-tuning with the original tokenizer. If SELFIES symbols are fragmented into generic subword pieces, the model can learn frequent input-output associations but is not operating over chemically meaningful units. EnzymeLM therefore first aligns the tokenizer with SELFIES, then builds molecule-level structure understanding through pretraining, and finally fine-tunes on reaction traces that combine biochemical reasoning with direct product-set supervision.

4 Results

We evaluate EnzymeLM on enzymatic forward reaction prediction without reactant names or EC number input at test time. The model receives only reactant SELFIES strings and must generate the full product set. For a ground-truth product set G and predicted product set P , **any-product accuracy** counts a match when $G \cap P \neq \emptyset$, while **all-products accuracy** requires exact set equality after canonicalization, $G = P$. All-products accuracy is the central metric because it penalizes missing co-products, missing cofactors, leaving groups, and hallucinated extra products. We also report primary-product accuracy, prediction validity, and Tanimoto similarity, where Tanimoto measures structural similarity between predicted and reference products. Although the final product set is the primary evaluated output, EnzymeLM is trained to produce a structured reaction trace before final generation; the trace exposes intermediate reaction-class, product-drafting, and atom-accounting claims that can be inspected during error analysis.

The results support three conclusions. First, prompted LLMs and existing chemistry-specific sequence models do not recover complete enzymatic product sets from reactant structures alone. Second, EnzymeLM substantially improves both partial and complete product recovery, with the largest gains on all-products accuracy. Third, remaining errors are not primarily molecular-syntax failures: invalid outputs are rare, and the main challenge is reaction-level product-set correctness.

Table 1: Main enzymatic forward prediction results. Open-source LLMs are evaluated with few-shot prompting after zero-shot prompting failed. Chemistry-specific models are evaluated in their native SMILES-based reaction formats. EnzymeLM receives reactant SELFIES strings without reactant names or EC number input at test time. Full Wilson and bootstrap confidence intervals for EnzymeLM variants are reported in Appendix F.1.

Model	Type	Any-Product	All-Products	Primary	Validity	Tanimoto
Gemma-3-4B, few-shot	General LLM	1.60	0.50	–	89.50	–
Gemma-3-27B, few-shot	General LLM	5.30	0.10	–	87.10	–
LLaMA-3.3-70B, few-shot	General LLM	2.30	0.30	–	71.70	–
Qwen3-80B, few-shot	General LLM	0.40	0.10	–	94.80	–
LlaSMol	Chemistry LLM	0.74	0.32	0.53	48.21	0.0774
ReactionT5v2	Reaction specialist	7.59	1.90	4.11	91.24	0.2437
Gemma direct SFT	EnzymeLM ablation	53.20	9.50	16.90	99.50	0.6261
Pretrain + reasoning-only SFT	EnzymeLM ablation	53.70	19.40	22.80	99.80	0.6654
Pretrain + reasoning/degradation SFT	EnzymeLM ablation	72.20	27.10	28.40	99.90	0.7533
Pretrain + SFT data mix	EnzymeLM	72.70	34.80	31.00	99.60	0.7707

Baseline models fail to recover complete enzymatic product sets. Table 1 compares EnzymeLM with open-source LLMs and chemistry-specific sequence models. For open-source LLMs, zero-shot prompting failed, so we report few-shot prompting with a strong system prompt and worked examples. These models were allowed to output either SELFIES or SMILES; outputs were decoded, canonicalized, and evaluated using the same molecular matching protocol. LlaSMol and ReactionT5v2 were evaluated using their native SMILES-based reaction formats.

The baseline gap is large. Gemma-3-27B gives the best prompted-LLM any-product result at 5.30%, but only 0.10% all-products accuracy. The best prompted-LLM all-products result is Gemma-3-4B at 0.50%. Chemistry-specific models improve format alignment but still fail on complete recovery: LlaSMol reaches 0.74% any-product and 0.32% all-products accuracy, while ReactionT5v2 reaches 7.59% and 1.90%. Thus, existing models occasionally recover a product fragment, but do not solve full product-set prediction from reactant structures alone.

EnzymeLM improves the strict full-product setting. The best EnzymeLM model, Pretrain + SFT data mix, achieves 72.70% any-product accuracy, 34.80% all-products accuracy, 31.00% primary-product accuracy, 99.60% validity, and 0.7707 macro Tanimoto similarity. Relative to ReactionT5v2, this is a 65.11-point gain in any-product accuracy and a 32.90-point gain in all-products accuracy. The any-product result shows that EnzymeLM usually identifies at least one correct product; the all-products result is more important because it shows that the model often recovers the complete product set. The high Tanimoto score indicates that many non-exact predictions remain structurally close to the reference products.

Ablations show that product-set recovery requires both molecular adaptation and mixed supervision. Direct SFT from Gemma already improves over prompted LLMs, reaching 53.20%

any-product and 9.50% all-products accuracy. This shows that reaction-level fine-tuning is necessary, but the large any-product to all-products gap indicates that direct SFT mostly learns partial product recovery. Adding molecular pretraining and reasoning-only SFT increases all-products accuracy to 19.40%, more than doubling direct SFT. Adding degraded self-correction further improves performance to 72.20% any-product and 27.10% all-products accuracy.

The best result comes from the mixed SFT curriculum, which reaches 34.80% all-products accuracy. Compared with reasoning/degradation SFT, the mixed curriculum gives only a small any-product gain but a larger all-products gain, from 27.10% to 34.80%. This suggests that reasoning traces help the model learn reaction constraints, while direct product-generation supervision improves final product-set completion. The confidence intervals support this interpretation: the best model’s all-products interval, [31.91, 37.81], is separated from the reasoning/degradation interval, [24.44, 29.94]. Macro Tanimoto also improves from 0.7533 to 0.7707.

Validation-test agreement and product-level similarity analysis support the result. Appendix F reports validation-test comparisons and product-level similarity analysis. The best model is stable across splits: all-products accuracy is 34.40% on validation and 34.80% on test; any-product accuracy is 71.40% and 72.70%; and macro Tanimoto is 0.7564 and 0.7707. These gaps are small relative to the improvements over baselines and weaker ablations.

Exact reactions are deduplicated across train, validation, and test splits, but the same product molecule can appear in multiple distinct enzymatic reactions. We therefore stratify performance by maximum similarity between each test reaction’s ground-truth products and the training-product set. The strict all-products result is not concentrated only in the exact-overlap subset: examples with exact product overlap achieve 33.01% all-products accuracy, while examples with maximum training-product similarity below 0.6 achieve 32.59%. In contrast, any-product accuracy is much higher in the exact-overlap subset (86.11% versus 45.09%), indicating that product recurrence mainly helps partial product recovery. This supports our use of all-products accuracy as the primary metric.

EC-stratified results reveal chemically meaningful variation. Appendix G reports EC-stratified performance. Transferases are strongest: EC 2 reaches 40.73% all-products accuracy, 80.13% any-product accuracy, and 0.8134 mean Tanimoto over 302 examples. EC 1 oxidoreductases also perform strongly, with 35.23% all-products accuracy and 0.7943 mean Tanimoto over 264 examples. Hydrolases are the weakest major class: EC 3 reaches 28.57% all-products accuracy, 53.11% any-product accuracy, and 0.7185 mean Tanimoto over 273 examples. Prediction validity remains above 99% in every EC class, indicating that class variation is driven by reaction correctness rather than molecular string validity.

Error analysis shows that syntax is mostly solved, but reaction accounting remains hard. Appendix F shows that invalid outputs account for only 0.40% of the test set. The dominant errors are atom-balance violations and wrong-main-product predictions: 34.80% of examples are exact full-product matches, 33.00% are atom-balance violations, and 27.40% are wrong-main-product failures. A complementary EC-stratified failure analysis shows that most all-products failures involve missing and hallucinated side products together (56.13% of failures) or wrong main product/no product match (42.94%). These results show that EnzymeLM’s main limitation is not invalid SELFIES generation, but assigning the correct atoms to the correct complete product set.

Qualitative generations illustrate the structured output format. Appendix H shows representative prompt/response generations. The model receives a SELFIES-only prompt and emits a structured response containing reactant identification, EC-class inference, product drafting, atom inventories, balance verification, and final product SELFIES. These examples show how the output exposes intermediate model claims before final product generation, making product-set errors easier to localize than in products-only generation.

Overall, the results show a clear progression. Prompted LLMs and existing chemistry-specific sequence models fail on complete product-set prediction. Direct SFT learns partial reaction behavior but remains weak on all-products accuracy. Molecular pretraining, reasoning supervision, degraded self-correction, and mixed product-generation targets progressively improve complete product recovery. The post-hoc analyses show that the result is stable across validation and test splits, product recurrence primarily affects partial recovery, and EC-stratified and failure-mode analyses make the remaining errors diagnostically clear.

5 Discussion & Conclusion

EnzymeLM suggests that enzymatic reaction prediction is better framed as structured reaction understanding for balanced product-set generation, rather than ordinary sequence translation. The final output is still the canonicalized product set, but the model is trained to reach it through reaction-class hypotheses, product drafts, atom inventories, balance checks, and final SELFIES generation. This structure matters because the central difficulty is not molecular-string validity: SELFIES largely addresses that, and the best EnzymeLM model reaches 99.60% valid predictions. The harder problem is reaction accounting: deciding which atoms belong in which products, which cofactors or small molecules must appear, and when a plausible main product is insufficient because the full product set is incomplete.

This distinction explains why prompted LLMs and existing chemistry-specific sequence models underperform. Such models may generate chemical-looking strings or learn frequent transformation patterns, but balanced enzymatic prediction requires product-set completeness under stoichiometric constraints. EnzymeLM improves by aligning the token space with chemically meaningful SELFIES units, building molecule-level familiarity through pretraining, and fine-tuning on reaction traces that expose product drafting and atom accounting before final generation. The gain therefore reflects better alignment between the model’s representation, supervision, and the formal constraints of the task.

The ablations support this interpretation. Direct reaction SFT improves substantially over prompted LLMs, but remains weak on all-products accuracy, indicating that reaction supervision alone mostly teaches partial product recovery. Molecular pretraining, reasoning-only SFT, and degraded self-correction progressively improve complete product-set prediction. The best result comes from the mixed SFT curriculum, where structured traces are paired with direct product-generation targets. This suggests that structured reasoning is most useful as a training scaffold and diagnostic output, not as a substitute for supervision on the final structured object. The traces expose intermediate model claims, but they should not be treated as independent proof of the true enzymatic mechanism.

The product-level similarity analysis clarifies what the benchmark measures. Exact reactions are deduplicated across train, validation, and test, but product molecules can recur across distinct enzymatic reactions. The stratified results show that exact product recurrence mainly affects partial product recovery: any-product accuracy is much higher in the exact-overlap subset, while all-products accuracy is comparable between exact-overlap and lower-similarity examples. This supports all-products accuracy as the primary metric because it requires the complete canonicalized product set, with no missing or extra valid products.

The EC-stratified results show that balanced enzymatic prediction is not uniform across enzyme families. Transferases are the clearest strength, while hydrolases remain more difficult, especially when cleavage products and co-products must be identified exactly. This variation matters because aggregate accuracy can hide chemically meaningful differences across reaction classes. Future enzymatic reaction models should therefore be evaluated not only by global exact match, but also by EC class, product-set size, cofactor involvement, stereochemical complexity, and molecular novelty.

A natural next step is enzymatic retrosynthesis. The present results also show why retrosynthesis is more difficult than simply reversing the forward model. Forward prediction already requires complete product-set accounting, while retrosynthesis is explicitly one-to-many: a target molecule may be reachable through many precursor sets, enzyme classes, and multi-step routes. EnzymeLM provides one component of such a system: a forward model that can evaluate whether proposed reactants plausibly yield a target product set. Combined with retrosynthetic proposal generation and pathway search, this could support discovery of novel enzyme-mediated synthesis routes.

More broadly, EnzymeLM argues for a domain-invariant view of scientific language modeling. The result is not that scale is irrelevant; larger chemistry-native models may improve substantially. Rather, scale without domain alignment is insufficient for formal scientific languages such as molecular strings. In enzymatic chemistry, useful models must respect molecular grammar, graph equivalence, atom conservation, product-set completeness, and biochemical class structure. EnzymeLM shows that a compact model can make substantial progress when its tokenizer, pretraining tasks, and fine-tuning objectives are aligned with these constraints.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] K Singhal, S Azizi, T Tu, SS Mahdavi, J Wei, HW Chung, N Scales, A Tanwani, H Cole-Lewis, S Pfohl, et al. Large language models encode clinical knowledge. *arxiv. arXiv preprint arXiv:2212.13138*, 2022.
- [3] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023.
- [4] Daniil A Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023.
- [5] Andres M Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. Chemcrow: Augmenting large-language models with chemistry tools. *arXiv preprint arXiv:2304.05376*, 2023.
- [6] Mark A Huffman, Anna Fryszkowska, Oscar Alvizo, Margie Borra-Garske, Kevin R Campos, Keith A Canada, Paul N Devine, Da Duan, Jacob H Forstater, Shane T Grosser, et al. Design of an in vitro biocatalytic cascade for the manufacture of islatravir. *Science*, 366(6470):1255–1259, 2019.
- [7] Silvia Donzella and Martina Letizia Contente. The joint effort of enzyme technology and flow chemistry to bring biocatalytic processes to the next level of sustainability, efficiency and productivity. *Journal of Flow Chemistry*, 14(1):85–96, 2024.
- [8] Daniel Probst, Matteo Manica, Yves Gaetan Nana Teukam, Alessandro Castrogiovanni, Federico Paratore, and Teodoro Laino. Biocatalysed synthesis planning using data-driven learning. *Nature communications*, 13(1):964, 2022.
- [9] Esther Heid, Daniel Probst, William H Green, and Georg KH Madsen. Enzymemap: curation, validation and data-driven prediction of enzymatic reactions. *Chemical Science*, 14(48):14229–14242, 2023.
- [10] David Weininger. Smiles, a chemical language and information system. 1. introduction to methodology and encoding rules. *Journal of chemical information and computer sciences*, 28(1):31–36, 1988.
- [11] Mario Krenn, Florian Häse, AkshatKumar Nigam, Pascal Friederich, and Alan Aspuru-Guzik. Self-referencing embedded strings (selfies): A 100% robust molecular string representation. *Machine Learning: Science and Technology*, 1(4):045024, 2020.
- [12] Sébastien Moretti, Van Du T Tran, Florence Mehl, Mark Ibberson, and Marco Pagni. Metanetx/mnxref: unified namespace for metabolites and biochemical reactions in the context of metabolic models. *Nucleic acids research*, 49(D1):D570–D574, 2021.
- [13] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [14] Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 4, 2025.
- [15] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [16] Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. Chemberta: large-scale self-supervised pretraining for molecular property prediction. *arXiv preprint arXiv:2010.09885*, 2020.

- [17] J Ross, B Belgodere, V Chenthamarakshan, I Padhi, Y Mroueh, and P Das. Large-scale chemical language representations capture molecular structure and properties, arxiv. *arXiv preprint arXiv:2106.09553*, 2021.
- [18] Zequn Liu, Wei Zhang, Yingce Xia, Lijun Wu, Shufang Xie, Tao Qin, Ming Zhang, and Tie-Yan Liu. Molxpt: Wrapping molecules with text for generative pre-training. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1606–1616, 2023.
- [19] Tatsuya Sagawa and Ryosuke Kojima. Reaction5: a pre-trained transformer model for accurate chemical reaction prediction with limited data. *Journal of Cheminformatics*, 17(1):126, 2025.
- [20] Botao Yu, Frazier N Baker, Ziqi Chen, Xia Ning, and Huan Sun. Llasmol: Advancing large language models for chemistry with a large-scale, comprehensive, high-quality instruction tuning dataset. *arXiv preprint arXiv:2402.09391*, 2024.
- [21] Prathamesh Kalamkar, Ned Letcher, Meissane Chami, Sahger Lad, Shayan Mohanty, and Prasanna Pendse. The tokenization bottleneck: How vocabulary extension improves chemistry representation learning in pretrained language models. *arXiv preprint arXiv:2511.14365*, 2025.
- [22] Miguelangel Leon, Yuriy Perezhohin, Fernando Peres, Aleš Popovič, and Mauro Castelli. Comparing smiles and selfies tokenization for enhanced chemical language modeling. *Scientific Reports*, 14(1):25016, 2024.
- [23] Alexius Wadell, Anoushka Bhutani, and Venkatasubramanian Viswanathan. Tokenization for molecular foundation models. *Journal of Chemical Information and Modeling*, 66(3):1384–1393, 2026.
- [24] Atakan Yüksel, Erva Ulusoy, Atabey Ünlü, and Tunca Doğan. Selformer: molecular representation learning via selfies language models. *Machine Learning: Science and Technology*, 4(2):025035, 2023.
- [25] Shuhan Guo, Yatao Bian, Ruibing Wang, Nan Yin, Zhen Wang, and Quanming Yao. Unimot: Unified molecule-text language model with discrete token representation. *arXiv preprint arXiv:2408.00863*, 2024.
- [26] Wen Tao, Jing Tang, Alvin Chan, Bryan Hooi, Baolong Bi, Nanyun Peng, Yuansheng Liu, and Yiwei Wang. How to make large language models generate 100% valid molecules? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 26576–26591, 2025.
- [27] Daniel Probst, Matteo Manica, Yves Gaëtan Nana Teukam, Alessandro Castrogiovanni, Federico Paratore, and Teodoro Laino. Molecular transformer-aided biocatalysed synthesis planning, 2021.
- [28] Chenqing Hua, Bozitao Zhong, Sitao Luan, Liang Hong, Guy Wolf, Doina Precup, and Shuangjia Zheng. Reactzyme: A benchmark for enzyme-reaction prediction. *Advances in Neural Information Processing Systems*, 37:26415–26442, 2024.
- [29] Zhenkun Shi, Jun Zhu, Dehang Wang, BoYu Chen, Qianqian Yuan, Zhitao Mao, Fan Wei, Weining Wu, Xiaoping Liao, and Hongwu Ma. Rxnrecer enables fine-grained enzymatic function annotation through active learning and protein language models. *arXiv preprint arXiv:2603.12694*, 2026.
- [30] Wenjia Qian, Xiaorui Wang, Yu Kang, Peichen Pan, Tingjun Hou, and Chang-Yu Hsieh. A general model for predicting enzyme functions based on enzymatic reactions. *Journal of Cheminformatics*, 16(1):38, 2024.
- [31] Tiantao Liu, Jiangcheng Xu, Xinke Zhan, Shaolong Lin, and Shirley WI Siu. Enzyformer: a two-stage pretrained model for enzymatic retrosynthesis. *Journal of Cheminformatics*, 2026.
- [32] Xuan Liu, Hongxiang Li, and Huimin Zhao. Chemoenzymatic synthesis planning guided by synthetic potential scores. *Digital Discovery*, 4(9):2534–2547, 2025.

- [33] Mohit Anand, Vikas Upadhyay, and Costas D Maranas. minchembio: expanding chemical synthesis with chemo-enzymatic pathways using minimal transitions. *ACS Synthetic Biology*, 14(3):756–770, 2025.
- [34] Vikas Upadhyay, Mohit Anand, and Costas D Maranas. novostoic2. 0: An integrated framework for pathway synthesis, thermodynamic evaluation, and enzyme selection. *PLoS computational biology*, 21(8):e1012516, 2025.

A Limitations

EnzymeLM is trained and evaluated on ECReact, so its performance should be interpreted within the coverage, curation choices, and split protocol of that dataset. Exact reactions are deduplicated across train, validation, and test, but the same metabolite can appear as a product in multiple distinct enzymatic reactions. We therefore analyze product-level similarity in Appendix F; the analysis shows that exact product recurrence mainly affects any-product accuracy, while strict all-products accuracy remains comparable between exact-overlap and lower-similarity examples. Evaluation on additional curated resources such as EnzymeMap, ReactZyme-style splits, and product- or reaction-family-disjoint benchmarks would further test out-of-distribution generalization.

The current model isolates chemistry-native reactant-to-product prediction: at test time, it receives reactant SELFIES but no reactant names, EC number, enzyme sequence, active-site structure, organism, pH, temperature, or cellular context. This makes the benchmark deliberately strict for molecular-structure-based prediction, but it does not determine whether a predicted transformation is feasible for a specific enzyme or condition. Protein-conditioned systems such as RXNRECer address a complementary problem by predicting enzyme-catalyzed reactions from protein sequences rather than predicting products from reactant structures. Finally, stoichiometric balance and exact product matching do not fully characterize enzymatic correctness: SELFIES improves molecular validity and the structured traces expose intermediate model claims, but future work should add enzyme-conditioned prediction, stereochemistry- and charge-aware evaluation, larger chemistry-native backbones, retrosynthesis, and multi-step pathway search.

Potential societal impacts. EnzymeLM could support greener and more efficient biocatalytic synthesis by improving computational tools for enzymatic reaction understanding, product-set prediction, and future pathway design. Such models may help researchers explore enzyme-mediated routes that reduce synthetic steps, reagent use, and waste. At the same time, enzymatic reaction models should be used with domain expertise and experimental validation, since predicted reactions may be chemically plausible but biologically infeasible under real enzyme, organism, or process conditions. As with other machine-learning tools for chemistry and biology, potential risks include overreliance on model outputs, propagation of dataset biases, inequitable access to high-quality computational tools, and possible misuse for designing harmful biochemical transformations. Responsible deployment should therefore pair model predictions with expert review, provenance tracking, safety screening, and validation in controlled research settings.

B Implementation and Training Details

B.1 Tokenizer Extension

EnzymeLM uses a merged tokenizer constructed by extending the base Gemma tokenizer with SELFIES tokens. The tokenizer merge script loads the base tokenizer and a SELFIES tokenizer, extracts bracketed SELFIES tokens, and adds any tokens not already present in the base vocabulary. The original Gemma special tokens are preserved, and the padding token is set to the EOS token when no explicit padding token is available. The resulting tokenizer is saved and used for both molecular pretraining and reaction fine-tuning.

B.2 Molecular Pretraining

Molecular pretraining data are constructed from MetaNetX `chem_prop.tsv`. Each valid compound must contain a name, formula, and SMILES string. SMILES are parsed with RDKit and converted into SELFIES. Invalid or unencodable compounds are removed.

For each valid molecule, the preprocessing script generates up to five distinct SELFIES enumerations. The canonical SMILES representation is encoded first, followed by randomized SMILES enumerations until the target number of SELFIES variants is reached or the maximum number of attempts is exhausted. From these enumerations, three task families are generated:

- **Isomorphism:** one SELFIES enumeration is mapped to another enumeration of the same molecule.
- **Nomenclature:** the compound name is mapped to a SELFIES representation.
- **Stoichiometry:** a SELFIES representation is mapped to the molecular formula.

The compound list is shuffled with seed 42 and split into 80% train, 10% validation, and 10% test. Molecular pretraining uses pure autoregressive next-token prediction over the serialized task sequences. The implementation assigns labels to all input tokens, so no completion-only masking is used during this stage. Sequences are packed to a maximum length of 512 tokens.

The pretraining script uses bf16 precision, optional LoRA, gradient checkpointing, and Hugging Face Trainer. When LoRA is enabled, the pretraining configuration uses rank $r = 64$, $\alpha = 128$, target modules set to `all-linear`, and saves `embed_tokens` and `lm_head` with the adapter. The default optimizer configuration is Hugging Face Trainer’s AdamW setup unless overridden. The script default learning rate is 2×10^{-4} , default maximum sequence length is 512, default per-device batch size is 4, and default gradient accumulation is 8. In the documented distributed pretraining command, training is launched with 4 processes, bf16 precision, per-device batch size 16, gradient accumulation 1, gradient checkpointing, and LoRA. We use 4 A5000 GPUs.

C Reaction SFT Data Generation

Reaction fine-tuning data are generated from ECR react reactions represented in SELFIES format. The SFT data generation script grounds reactions using MetaNetX compound names, ECR react metadata, ExPASy EC hierarchy files, and RDKit-derived molecular graphs.

For compound name grounding, the script builds an InChIKey-based lookup from MetaNetX `chem_prop.tsv`. Each compound SMILES is parsed with RDKit and converted to an InChIKey. The script stores both full InChIKey matches and skeleton-level matches, allowing compound names to be recovered when exact stereochemical or protonation forms differ.

For each reaction, reactant and product SELFIES are decoded to SMILES and parsed with RDKit. Atom inventories are computed after adding explicit hydrogens. The reactant-side and product-side atom inventories are then serialized directly into the reasoning trace.

C.1 EC Mapping

EC annotations are assigned through a cascaded lookup procedure. The EC mapping script builds lookup tables from ECR react, MetaNetX reaction properties, ExPASy `enzclass.txt`, and ExPASy `enzyme.dat`. The cascade proceeds from higher-precision to higher-coverage matching:

1. exact reactant-product set match,
2. core molecule match after removing small molecules and requiring sufficient heavy atoms,
3. largest reactant and largest product pair match,
4. substrate-level match using the most frequent EC assignment for a matched molecule,
5. heuristic Level 1 EC fallback when no lookup match is available.

When a specific EC mapping is available, the trace includes a three-level EC class and a human-readable hierarchy description. When EC4 catalog information is available, the mechanism text can use the cataloged catalyzed reaction. Otherwise, class-specific templates are used to describe the transformation. When no EC lookup is found, the script assigns a heuristic Level 1 EC class such as oxidoreductase, transferase, hydrolase, lyase, isomerase, ligase, or translocase.

C.2 Chain-of-Thought Trace Construction

Each SFT trace is generated automatically. No reasoning traces are manually written. A standard trace contains:

1. reactant identification,
2. EC classification,
3. optional mechanism description,
4. product drafting,
5. reactant atom inventory,
6. product atom inventory,
7. balance verification,
8. final product SELFIES.

The trace is wrapped inside a `<think>` block, followed by the final product SELFIES. The EC class is included only in the target trace, not in the input prompt. Thus, during inference, the model must infer the enzymatic class from the reactants rather than receiving it externally.

C.3 Degraded CoT Augmentation

To train self-correction, 20% of traces are deliberately corrupted at the intermediate atom-inventory stage. The script randomly selects one or two atom types from the product inventory and reduces their counts when possible. The trace then marks the reaction as imbalanced, reports the atom-count delta, and adds a correction step restoring the correct product inventory before emitting the final product SELFIES:

draft products $\rightarrow$ incorrect inventory $\rightarrow$ imbalance detected $\rightarrow$ corrected inventory $\rightarrow$ final SELFIES.

This construction teaches the model to use atom balance as an operational constraint rather than treating the inventory as decorative text.

C.4 Reaction-Level Augmentation

Each ECR react training reaction is expanded into 10 augmented samples. For each augmentation, the script randomly permutes reactant and product order and generates randomized SELFIES enumerations from the underlying RDKit molecules. Prompt format is sampled with a 50/50 mixture:

- **Hybrid prompt:** compound names plus reactant SELFIES structures.
- **Structure-only prompt:** reactant SELFIES structures only.

This augmentation makes the model robust to reaction participant ordering and alternative SELFIES serializations, while also grounding common chemical names to molecular structures.

D Reaction Fine-Tuning Details

Reaction fine-tuning uses completion-only supervised fine-tuning. Each example is represented as a chat-style user message paired with a model response. The tokenizer chat template is applied to the user message, and the model response is appended with an EOS token. Prompt tokens are assigned label `-100`, so the loss is computed only over the generated target portion.

The SFT corpus supports a 60/20/20 data mixture:

- 60% prompt-to-CoT-and-products, where the model generates the full reasoning trace followed by product SELFIES;
- 20% prompt-plus-CoT-to-products, where the reasoning trace is included in the prompt and the model generates only final product SELFIES;

- 20% prompt-to-products-only, where the model directly generates final product SELFIES.

The SFT script uses bf16 precision, gradient checkpointing, dynamic padding with `DataCollatorForSeq2Seq`, and optional LoRA. When LoRA is enabled, the default SFT configuration uses rank $r = 32$, $\alpha = 64$, dropout 0.05, and targets the projection and MLP modules:

`{q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj}`.

The script default learning rate is 1×10^{-5} , maximum sequence length is 1024, and training runs for 3 epochs. The documented distributed SFT command uses 8 processes, bf16 precision, per-process batch size 4, gradient accumulation 4, gradient checkpointing, and LoRA, giving an effective global batch size of 128.

E Evaluation Details

Evaluation is performed with greedy generation by default. The evaluation script uses batch size 8, maximum generation length 1024, one return sequence, one beam, temperature 1.0, and repetition penalty 1.1. Since the default number of return sequences is one, sampling is disabled and generation is deterministic.

Model outputs are parsed by removing special tokens and extracting the final prediction after the `</think>` tag. If no `</think>` tag is present, the full generated text is treated as the prediction.

E.1 Canonicalization and Product Matching

Predicted and ground-truth product strings are split on the molecular separator `..`. Each molecule is interpreted as SELFIES when possible and decoded to SMILES. If SELFIES decoding fails, the string is treated as SMILES directly. Molecules are parsed with RDKit and converted to canonical isomeric SMILES. Invalid molecules are discarded.

For each reaction, let G be the set of canonical ground-truth products and P be the set of canonical predicted products. Any-product accuracy is counted as correct when

$$G \cap P \neq \emptyset.$$

All-products accuracy is counted as correct when the predicted set exactly matches the ground-truth set:

$$P = G.$$

The implementation enforces both no missing products and no extra products by requiring that the number of matched ground-truth products equals $|G|$ and that $|P| = |G|$.

Primary-product accuracy is computed by taking the first molecule in the ground-truth product string as the primary product and checking whether its canonical form appears in the predicted product set.

E.2 Validity and Tanimoto Similarity

Prediction validity is computed at the reaction level as whether the prediction yields at least one valid canonical product molecule. Tanimoto similarity is computed using RDKit Morgan fingerprints with radius 2 and fingerprint size 2048, corresponding to ECFP4-style fingerprints.

Exact canonical product matches receive Tanimoto similarity 1.0. For remaining unmatched ground-truth and predicted products, the evaluation greedily assigns each unmatched ground-truth product to the available predicted product with the highest Tanimoto similarity. Unmatched ground-truth products receive similarity 0.0. The reaction-level Tanimoto score is averaged over the number of ground-truth products, and the final reported score is macro-averaged across reactions.

F Post-hoc Robustness and Error Analysis

We perform several post-hoc analyses on the completed validation and test outputs. No model retraining is performed. These analyses quantify uncertainty, compare validation and test performance, assess product-level similarity to the training set, and characterize the dominant failure modes.

F.1 Confidence Intervals

For binomial metrics, we report Wilson 95% confidence intervals over the 1,000-reaction test set. For macro Tanimoto similarity, we report percentile bootstrap confidence intervals using 10,000 bootstrap samples over test reactions. Table 2 summarizes the uncertainty estimates for the evaluated EnzymeLM variants.

Table 2: Test-set performance with 95% confidence intervals. Wilson intervals are used for binomial metrics; bootstrap percentile intervals are used for macro Tanimoto.

Model	All-Products	Any-Product	Primary	Validity	Tanimoto
Gemma direct SFT	9.50 [7.83, 11.48]	53.20 [50.10, 56.27]	16.90 [14.70, 19.35]	99.50 [98.83, 99.79]	0.6261 [0.6109, 0.6411]
Pretrain + reasoning-only SFT	19.40 [17.07, 21.97]	53.70 [50.60, 56.77]	22.80 [20.31, 25.50]	99.80 [99.27, 99.95]	0.6654 [0.6500, 0.6812]
Pretrain + reasoning/degradation SFT	27.10 [24.44, 29.94]	72.20 [69.34, 74.89]	28.40 [25.69, 31.27]	99.90 [99.44, 99.98]	0.7533 [0.7388, 0.7680]
Pretrain + SFT data mix	34.80 [31.91, 37.81]	72.70 [69.86, 75.37]	31.00 [28.21, 33.93]	99.60 [98.98, 99.84]	0.7707 [0.7559, 0.7852]

The confidence intervals show that the best model, Pretrain + SFT data mix, is clearly separated from the weaker ablations on all-products accuracy and Tanimoto similarity. This supports the conclusion that the gain is not a small fluctuation of the test set.

F.2 Validation-Test Agreement

To check whether test performance is consistent with validation performance, we compare the final validation and test metrics for each model. Table 3 reports the results for the best-performing model.

Table 3: Validation-test agreement for the best model, Pretrain + SFT data mix. Small validation-test gaps suggest that the final test performance is not a validation-specific artifact.

Metric	Validation	Test	Test - Val
All-products accuracy	34.40	34.80	0.40
Any-product accuracy	71.40	72.70	1.30
Primary-product accuracy	32.20	31.00	-1.20
Pass@1 exact	34.40	34.80	0.40
Prediction validity	99.60	99.60	0.00
Macro Tanimoto	0.7564	0.7707	0.0143
Elemental penalty	17.14	16.97	-0.17

The validation and test metrics are closely aligned. In particular, all-products accuracy differs by only 0.40 percentage points, while any-product accuracy differs by 1.30 points. This suggests that the reported test result is consistent with validation-set behavior.

F.3 Product-Level Similarity Stratification

To assess how recurring product molecules affect evaluation, we compute the maximum Tanimoto similarity between each test reaction’s ground-truth products and the training-product set. Exact reactions are deduplicated across train, validation, and test, but the same product molecule can appear in multiple distinct enzymatic reactions. We therefore stratify performance by ground-truth maximum training-product similarity.

Table 4: Performance stratified by maximum Tanimoto similarity between ground-truth test products and the training-product set. Exact product recurrence increases partial product recovery, while strict all-products accuracy remains comparable between exact-overlap and low-similarity examples.

GT max train similarity	n	All	All-correct	Any	Tanimoto
1.0	612	33.01	202	86.11	0.7971
[0.0, 0.6)	224	32.59	73	45.09	0.6767
[0.6, 0.8)	149	46.98	70	62.42	0.8105
[0.8, 1.0)	15	20.00	3	40.00	0.7013

The strict all-products metric is not dominated by exact product recurrence: reactions with exact training-product overlap achieve 33.01% all-products accuracy, while reactions with maximum training-product similarity below 0.6 achieve 32.59%. Exact recurrence mainly affects any-product accuracy, which is 86.11% in the exact-overlap subset and 45.09% in the low-similarity subset. This decomposition supports using all-products accuracy as the primary metric for complete reaction recovery.

F.4 Structured Failure Analysis

We categorize each test example according to the first failure mode relative to all-products exact match. Table 5 reports the structured error analysis for the best model.

Table 5: Structured error analysis for the best model, Pretrain + SFT data mix. Percent of failures is computed after excluding correct all-products predictions.

Category	Count	% of all	% of failures
Correct	348	34.80	–
Atom-balance violation	330	33.00	50.61
Wrong main product	274	27.40	42.02
Hallucinated extra product	26	2.60	3.99
Missing product	18	1.80	2.76
Invalid output	4	0.40	0.61

Invalid outputs are rare, accounting for only 0.40% of test examples. The dominant remaining errors are atom-balance violations and wrong-main-product predictions. This suggests that the primary remaining challenge is not molecular syntax, but reaction-level chemical correctness and complete stoichiometric accounting.

G EC-Stratified Analysis

We further stratify EnzymeLM performance by Enzyme Commission (EC) class for the best model, Pretrain + SFT data mix. This analysis uses the completed evaluation outputs for the 1,000-example ECReact test set and the corresponding mixed SFT training set. Training examples without parsed EC labels are excluded from the training distribution analysis.

G.1 Training EC Distribution

The mixed SFT training set contains 234,601 examples with parsed EC labels. The distribution is concentrated in EC 1, EC 2, and EC 3, which together account for 83.73% of labeled training examples. This imbalance should be considered when interpreting per-class performance, since EC 4, EC 5, EC 6, and especially EC 7 are much less represented.

Table 6: Training-set EC distribution for the mixed SFT corpus. Percentages are computed over examples with parsed EC labels.

EC Class	Count	Percent
EC 1	67,603	28.82
EC 3	66,808	28.48
EC 2	62,014	26.43
EC 5	15,431	6.58
EC 6	13,898	5.92
EC 4	8,814	3.76
EC 7	33	0.01

G.2 Accuracy by EC Class

Table 7 reports accuracy metrics by EC class. All-products accuracy is strict: the canonicalized predicted product set must exactly equal the canonicalized ground-truth product set, with no missing or extra valid products.

Table 7: EC-stratified accuracy for EnzymeLM on the ECReact test set. EC 2 transferases are the strongest class by all-products accuracy, while EC 3 hydrolases are the weakest major class.

EC Class	Test n	All-Products Acc.	Failure Rate	Any-Product Acc.	Valid Prediction
EC 2	302	40.73	59.27	80.13	99.67
EC 4	39	35.90	64.10	69.23	100.00
EC 1	264	35.23	64.77	87.88	99.62
EC 6	65	33.85	66.15	75.38	100.00
EC 5	57	31.58	68.42	56.14	100.00
EC 3	273	28.57	71.43	53.11	99.27

The model performs best on EC 2 transferases by strict all-products accuracy. EC 3 hydrolases are the weakest major class by all-products accuracy and any-product accuracy, suggesting that the model has greater difficulty identifying complete cleavage products and co-products for this class.

G.3 Tanimoto Similarity by EC Class

Table 8 reports EC-stratified Tanimoto similarity. Tanimoto similarity is averaged over ground-truth products using best predicted-product matches. Exact all-products accuracy is stricter than Tanimoto because Tanimoto can remain high when a co-product is missing or an extra product is hallucinated.

Table 8: EC-stratified Tanimoto similarity. The low zero-similarity rate indicates that most failures still produce chemically related products.

EC Class	Test n	Mean Tanimoto	Median Tanimoto	Exact 1.00	≥ 0.90	Zero
EC 2	302	0.8134	0.8651	41.06	46.36	0.33
EC 1	264	0.7943	0.8297	35.61	40.15	0.38
EC 6	65	0.7705	0.8182	35.38	46.15	0.00
EC 4	39	0.7243	0.7283	35.90	35.90	0.00
EC 3	273	0.7185	0.7344	28.57	33.70	0.73
EC 5	57	0.7173	0.7703	31.58	35.09	0.00

The gap between mean Tanimoto and all-products accuracy shows that many errors are near misses rather than unrelated predictions. In particular, the low zero-similarity rate across EC classes indicates that failed exact matches often remain chemically related to the reference products.

G.4 Failure Categories

Table 9 summarizes failure categories among examples where all-products exact match fails. The dominant failure mode is not invalid molecular generation. Instead, failures mostly involve incorrect product-set composition, especially combinations of missing and hallucinated products or wrong main products.

Table 9: Failure categories among all-products failures for the best EnzymeLM model.

Failure Category	Count	Percent of Failures
Missing and hallucinated products	366	56.13
Wrong main product or no product match	280	42.94
Invalid output	4	0.61
Hallucinated extra products only	2	0.31

Overall, EC-stratified analysis shows that EnzymeLM generalizes unevenly across enzyme families. Transferases are the clearest strength, while hydrolases and isomerases remain more challenging. Across classes, prediction validity remains above 99%, suggesting that the main remaining challenge is product-set correctness rather than generating valid molecular strings.

H Qualitative Examples

We provide representative prompt/response examples from EnzymeLM. Each example shows the model input, the structured response, and the final product SELFIES. Long SELFIES strings are shortened with \ldots for readability; raw prompts and complete generated outputs are retained in the released prediction files.

Table 10: Qualitative example: oxidoreductase with oxygen incorporation.

Prompt	Predict the full mass-balanced reaction: [O] [=C] [C] [=C] [C] ... [N] [C] [=Branch1] [C] [=O] ... [O] [=O] . [H+1]
Response	<think> STEP 1: IDENTIFICATION. Reactants: 9H-fluoren-9-one, NADH, O2, PMF. STEP 2: CLASSIFICATION. EC 1.14.18 - Oxidoreductases / Acting on paired donors, with incorporation or reduction of molecular oxygen. Mechanism: This oxidoreductase reaction acts on paired donors with incorporation of O2 of 9H-fluoren-9-one, using NADH as the electron carrier, yielding 1-Hydro-1,1a-dihydroxy-9-fluorenone, beta-nicotinamide adenine dinucleotide. STEP 3: DRAFTING. Target Products: 1-Hydro-1,1a-dihydroxy-9-fluorenone, beta-nicotinamide adenine dinucleotide. STEP 4: VERIFICATION. inventory_reactants: {C:34, N:7, O:17, P:2, H:38} inventory_products: {O:17, P:2, C:34, N:7, H:38} balanced: true </think> [O] [=C] [C] [=C] [C] ... [N] [C] [=Branch1] [C] [=O] ...

Table 11: Qualitative example: transferase-style product-set prediction.

Prompt	Predict the full mass-balanced reaction: [N] [C@@H1] [Branch1] [#Branch2] ... [N] [C] [C] [=Branch1] ... [N] [C] [=N] [C] [=N] ...
Response	<think> STEP 1: IDENTIFICATION. Reactants: L-phenylalanine, glycine, ATP. STEP 2: CLASSIFICATION. EC 2.7.2 - Transferases / Transferring phosphorus-containing groups / Phosphotransferases with a carboxy group as acceptor. Mechanism: This transferase reaction transfers a phosphorus-containing group from ATP to L-phenylalanine, yielding Phe-Gly, Adenosine_Diphosphate, polyphosphate. STEP 3: DRAFTING. Target Products: Phe-Gly, Adenosine_Diphosphate, polyphosphate. STEP 4: VERIFICATION. inventory_reactants: {H:32, P:3, O:17, C:21, N:7} inventory_products: {H:32, O:17, N:7, P:3, C:21} balanced: true </think> [N] [C@@H1] [Branch1] [#Branch2] ... [N] [C] [=N] [C] [=N] ... [O] [=P] [Branch1] [C] [O] ...

Table 12: Qualitative example: glycosidic hydrolase. This example is used as the representative case in the main paper.

Prompt	Predict the full mass-balanced reaction: [O] [C] [C] [Branch1] [C] [O] [C] [O] [C@@H1] [O]
Response	<think> STEP 1: IDENTIFICATION. Reactants: beta-D-apiofuranosyl-(1->6)-D-glucose, H2O. STEP 2: CLASSIFICATION. EC 3.2.1 - Hydrolases / Glycosylases / Glycosidases, i.e. enzymes hydrolyzing O- and S-glycosyl compounds. Mechanism: This hydrolase reaction cleaves glycosidic bonds in beta-D-apiofuranosyl-(1->6)-D-glucose using water, yielding beta-D-glucose, D-apiofuranose. STEP 3: DRAFTING. Target Products: beta-D-glucose, D-apiofuranose. STEP 4: VERIFICATION. inventory_reactants: {C:11, O:11, H:22} inventory_products: {H:22, O:11, C:11} balanced: true </think> [O] [C] [C@H1] [O] [C@@H1] [O] [C] [C] [Branch1] [C] [O] . . .

Table 13: Qualitative example: lyase-style generation.

Prompt	Predict the full mass-balanced reaction: [N] [C@@H1] [Branch1] [=Branch2] [N] [C] [=Branch1] [C] [=O] . . .
Response	<think> STEP 1: IDENTIFICATION. Reactants: L-histidine, D-glutamine. STEP 2: CLASSIFICATION. EC 4.1.1 - Lyases / Carbon-carbon lyases / Carboxy-lyases. Mechanism: This lyase reaction cleaves or forms bonds in L-histidine without hydrolysis or oxidation, yielding gamma-Glu-His, nitrogen atom. STEP 3: DRAFTING. Target Products: gamma-Glu-His, nitrogen atom. STEP 4: VERIFICATION. inventory_reactants: {O:5, H:19, C:11, N:5} inventory_products: {C:11, H:19, N:5, O:5} balanced: true </think> [N] [C@H1] [Branch2] [Ring1] [Branch2] [N]

Table 14: Qualitative example: ligase-class product-set generation.

Prompt	Predict the full mass-balanced reaction: [N] [C@@H1] [Branch1] [=Branch2] [N] [C@@H1] [Branch1] [Ring1] [O] [=O]
Response	<think> STEP 1: IDENTIFICATION. Reactants: L-histidine, L-cysteine, O2. STEP 2: CLASSIFICATION. EC 6.3.2 - Ligases / Forming carbon-nitrogen bonds / Acid-amino-acid ligases (peptide synthases). Mechanism: This ligase reaction joins L-histidine and L-cysteine, coupled to hydrolysis of ATP, yielding L-alanin-3-yl L-histidin-5-yl sulfoxide, H2O. STEP 3: DRAFTING. Target Products: L-alanin-3-yl L-histidin-5-yl sulfoxide, H2O. STEP 4: VERIFICATION. inventory_reactants: {S:1, H:16, O:6, C:9, N:4} inventory_products: {H:16, O:6, C:9, N:4, S:1} balanced: true </think> [N] [C@@H1] [Branch2] [Ring1] [#Branch1] [O]

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”,**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: All claims made in abstract clearly state claims made and supported by the paper.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Limitations section in appendix

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: No Theoretical result

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: All details in appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The code and data will be released upon acceptance as it goes beyond the 50mb limit in the supplementary material. A github link and a model link will be provided.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: All details are in appendix and main paper

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [\[Yes\]](#)

Justification: We provide bootstrapping CIs to make up for lack of seeds, which is standard in this area.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not include experiments.
- The authors should answer [\[Yes\]](#) if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [\[Yes\]](#)

Justification: Details in Appendix

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.

- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: We have read and adhered to code of ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Discussed in appendix.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper poses no such risks.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All licensed/other softwares have been cited.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: Assets cannot be provided as supplementary material. They will be released properly upon acceptance

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: No such experiments.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No such experiments

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: No LLMs used for research methodology.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.